Bất Đồng Bộ Với Future & Stream Trong Flutter

TRƯỜNG ĐẠI HỌC BÁCH KHOA — KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

Chủ đề 02: Báo cáo Tuần 02 — Môn: Lập trình Đa nền tảng

SVTH: Huỳnh Thiện Lộc – **MSSV:** 106230052
GVHD: TS. Nguyễn Duy Nhật Viễn

Đà Nẵng, 2026

Nội dung báo cáo

1. Phần I: Yêu Cầu Đề Bài & Bài Toán Thực Tế
   - Yêu cầu kỹ thuật & Phạm vi đề tài (Spec)
2. Phần II: Nền Tảng Bất Đồng Bộ & Vấn Đề Kỹ Thuật
   - Cơ chế Event Loop & Hàng đợi trong Dart
   - So sánh bản chất: Future vs Stream
   - Cơ chế xử lý tác vụ chờ với Future & async/await
3. Phần III: Thiết Kế & Hiện Thực Luồng Dữ Liệu Stream
   - Cấu trúc luồng với StreamController
   - Phản ứng giao diện: setState vs StreamBuilder
   - Xử lý vòng đời & Chống Memory Leak
4. Phần IV: Demo Thực Tế & Kết Quả Kiểm Thử
   - Sơ đồ luồng End-to-End từ Timer đến UI
   - Giao diện thực tế & 3 trạng thái điều khiển
   - Kiểm thử tự động tầng Stream (Unit Test)
   - Tài liệu tham khảo cụ thể

Phân công công việc (WBS)

TT	HỌ VÀ TÊN	NỘI DUNG ĐẢM NHẬN	TỶ LỆ
1	Huỳnh Thiện Lộc (106230052)	Nghiên cứu Event Loop, lập trình StreamController, xây dựng StreamBuilder, xử lý vòng đời dispose và viết Unit Test.	100%

Phần I: Yêu Cầu Đề Bài & Bài Toán Thực Tế

Yêu cầu kỹ thuật & Phạm vi đề tài (Spec)

Yêu Cầu Cốt Lõi (Core Specs)

- **Bất đồng bộ:** Hiểu cơ chế chờ tác vụ với `Future` và `async/await`.
- **Luồng thời gian thực:** Dùng `StreamController` phát dữ liệu và `StreamBuilder` cập nhật UI.
- **Cơ chế dọn dẹp:** Bắt buộc hủy (`cancel/close`) chống rò rỉ RAM (Memory Leak).

Yêu Cầu Demo & Tiêu Chí Đánh Giá

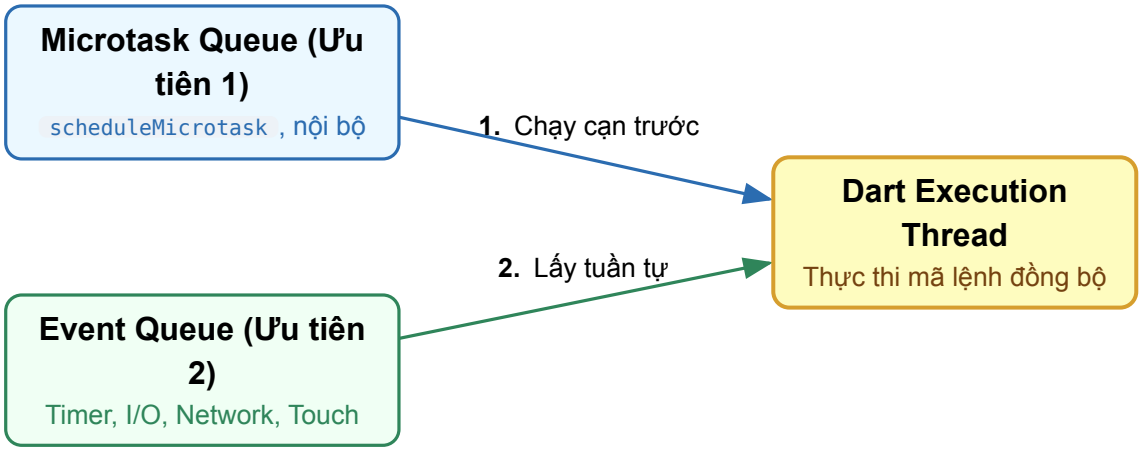
- **Giao diện động:** Hiển thị chỉ số biến đổi liên tục theo thời gian thực.
- **3 Nút tương tác:** Bắt đầu (Start), Tạm dừng (Pause), Đặt lại (Reset).
- **An toàn vòng đời:** Đảm bảo giải phóng sạch `Timer` và `Stream` khi thoát màn hình.

- **Mục tiêu giải pháp:** Không lạm dụng `setState()`, bảo đảm app đạt 60 FPS mượt mà và 0% Memory Leak.

Phần II: Nền Tảng Bất Đồng Bộ & Vấn Đề Kỹ Thuật

Cơ chế Event Loop & Hàng đợi trong Dart

- **Bản chất:** Dart chạy đơn luồng (Single Thread). Event Loop liên tục quét hai hàng đợi:



- **Future:** Đưa tác vụ chờ vào Event Queue, chỉ sinh **1 giá trị kết quả duy nhất**.

So sánh bản chất: Future vs Stream

Tiêu chí	Future (Single Value)	Stream (Continuous Values)
Số lượng dữ liệu	1 giá trị duy nhất (hoặc 1 lỗi)	Chuỗi nhiều giá trị phát theo thời gian
Mô hình trao đổi	Request - Response (Kéo dữ liệu)	Publisher - Subscriber (Đẩy sự kiện)
Vòng đời luồng	Tự đóng ngay khi hoàn thành	Mở liên tục, bắt buộc gọi `close()`
Widget UI tương ứng	`FutureBuilder`	`StreamBuilder`
Ứng dụng thực tế	Gọi REST API, đọc file, nạp database	Đồng hồ bấm giờ, cảm biến, WebSocket

- **Điểm mấu chốt:** `Stream` mở liên tục cho đến khi gọi `close()`, nếu quên hủy sẽ gây rò rỉ RAM.

Cơ chế xử lý tác vụ chờ với Future & async/await

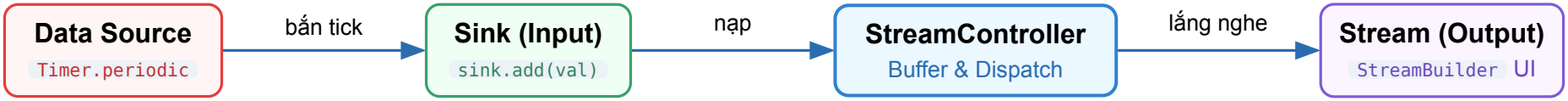
- **Bản chất của `await`:** Không khóa luồng UI (Non-blocking). Nó tạm dừng hàm hiện tại, nhường Dart Execution Thread cho các sự kiện khác trong Event Loop:

```dart
Future<String> fetchUserConfig() async {
  // 1. Gặp await: hàm tạm ngưng, giải phóng Thread cho khung hình 60 FPS
  final response = await http.get(Uri.parse('https://api.example.com/config'));
  // 2. Khi I/O hoàn tất, callback được đẩy vào Event Queue để chạy tiếp
  return response.body;
}
```

- **`async`:** Tự động đóng gói kiểu dữ liệu trả về thành một `Future<T>`.
- **`await`:** Mở gói giá trị từ `Future` một cách tuần tự, loại bỏ hoàn toàn hiện tượng Callback Hell (`.then()`).
- **Bắt lỗi (Error Handling):** Dùng `try-catch` đồng bộ quen thuộc để bắt lỗi mạng/timeout.

Phần III: Thiết Kế & Hiện Thực Luồng Dữ Liệu Stream

Cấu trúc luồng với StreamController




```dart
final _controller = StreamController<int>.broadcast();
Stream<int> get stream => _controller.stream; // Công khai Stream ra ngoài
void emitTick(int tick) {
  if (!_controller.isClosed) _controller.sink.add(tick); // Nạp vào Sink
}
```

- `.broadcast()`: Cho phép nhiều UI widget cùng lắng nghe đồng thời 1 nguồn Stream.
- `sink.add()`: Cổng nạp dữ liệu vào luồng; luôn kiểm tra `!isClosed` để tránh `Bad state`.

Phản ứng giao diện: setState vs StreamBuilder

- **Vấn đề của setState:** Mỗi nhịp Stream bắn về, `setState()` ép rebuild toàn bộ cây Widget gây drop FPS.
- **Giải pháp StreamBuilder:** Lắng nghe Stream và rebuild **duy nhất widget con bên trong**:

```
StreamBuilder<int>(
  stream: _service.stream,
  initialData: _service.currentCount,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    final value = snapshot.data ?? 0;
    return Text('$value', style: const TextStyle(fontSize: 48));
  },
)
```

- `snapshot.connectionState`: Quản lý 3 trạng thái chuẩn (`waiting`, `active`, `done`).

Xử lý vòng đời & Chống Memory Leak

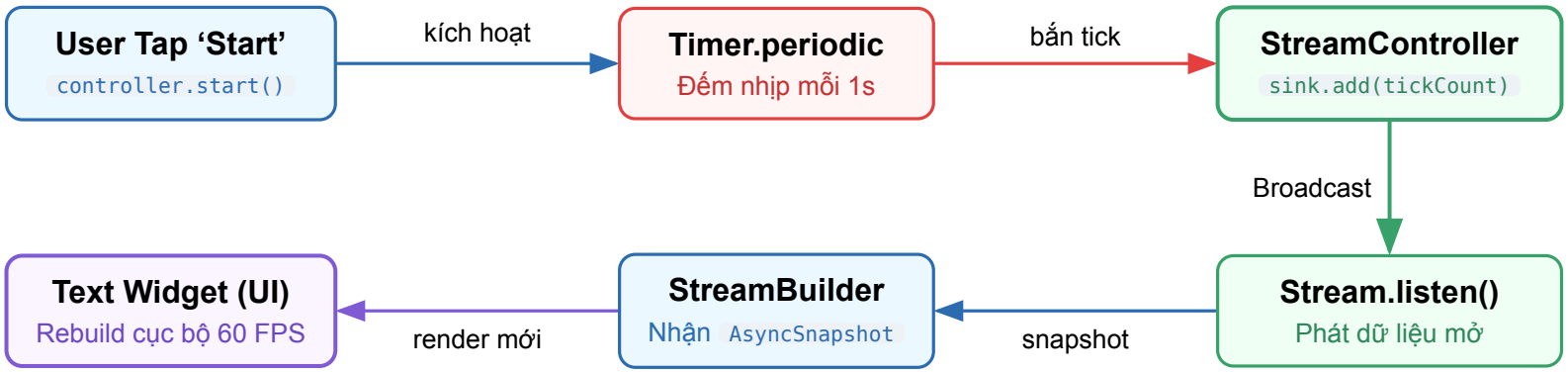
- **Nguy cơ:** Khi rời màn hình, `Timer` và `Stream` không hủy sẽ tiếp tục chạy ngầm, ngốn CPU và rò rỉ RAM.
- **Quy tắc dọn dẹp bắt buộc (Clean-up Pattern):** Override phương thức `dispose()`:

```
@override
void dispose() {
  _timer?.cancel();        // 1. Dừng ngay Timer phát sự kiện
  _controller.close();     // 2. Đóng hoàn toàn StreamController
  super.dispose();
}
```

- **Kiểm soát an toàn:** Luôn kiểm tra `!_controller.isClosed` trước khi gọi `sink.add()` để tránh crash lỗi `Bad state: Cannot add event after closing`.

Phần IV: Demo Thực Tế & Kết Quả Kiểm Thử

Sơ đồ luồng End-to-End từ Timer đến UI

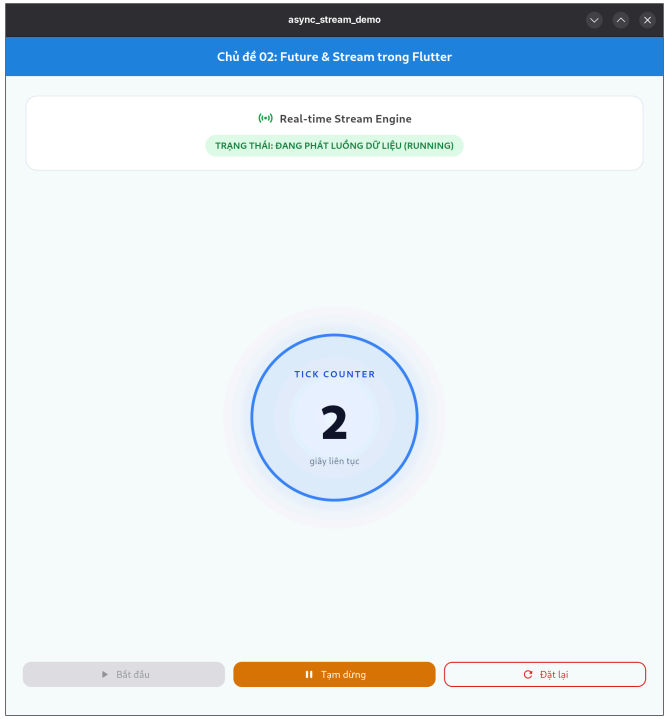


- **Nguyên tắc cô lập UI:** Chỉ widget nằm trong builder của `StreamBuilder` mới bị vẽ lại khi có dữ liệu mới.

Giao diện thực tế & 3 trạng thái điều khiển

Xử lý 3 trạng thái tương tác:

- **Bắt đầu (Start):** Kích hoạt Timer 1s, phát luồng nhịp đếm liên tục ra Stream.
- **Tạm dừng (Pause):** Hủy Timer `cancel()`, đóng băng giá trị hiển thị hiện thời.
- **Đặt lại (Reset):** Reset giá trị về 0 và phát sự kiện cập nhật giao diện ngay lập tức.
- **Thoát màn hình:** `dispose()` tự động giải phóng sạch `Timer` và `StreamController`.



Kiểm thử tự động tầng Stream (Unit Test)

- **Mục tiêu:** Kiểm tra luồng phát dữ liệu chuẩn xác theo thời gian bằng matcher `emitsInOrder`:

```
test('Start phát đúng chuỗi giá trị tăng dần theo thời gian', () async {
  service.start();

  // Dùng emitsInOrder kiểm tra chuỗi 3 giá trị liên tiếp từ Stream
  await expectLater(
    service.stream.take(3),
    emitsInOrder([1, 2, 3]),
  );
});
```

- **Kết quả:** Chạy `flutter test` kiểm thử độc lập logic mà không phụ thuộc vào giao diện hay máy ảo.

Tài liệu tham khảo cụ thể

1. **Google Flutter Documentation**, *Asynchronous Programming: Futures & Streams*:
   - Cơ chế Event Loop, Microtask Queue và cơ chế reactive của `StreamBuilder`.
   - https://docs.flutter.dev/libraries/architecture/async-programming
2. **Dart Language Specification**, *Stream & StreamController Library Guide*:
   - Phân biệt Single-subscription vs Broadcast Stream, quản lý `Sink` và `Stream`.
   - https://api.dart.dev/stable/dart-async/StreamController-class.html
3. **Effective Dart: Usage**, *Lifecycle Management & Cleaning Up Async Resources*:
   - Nguyên tắc bắt buộc giải phóng `Timer`, `cancel()` subscription và `close()` controller trong `dispose()`.
   - https://dart.dev/effective-dart/usage